

Reporte Técnico: Proyecto Final (Fase 3) - Procesador Segmentado MIPS de 32 bits

Arquitectura De Computadoras

Universidad de Guadalajara - Centro Universitario de Ciencias Exactas e Ingenierías (CUCEI)

Oscar Emiliano Ramírez Pérez

Alondra Estefania Hernández Vargas

1. Introducción

El diseño y la implementación de sistemas digitales complejos, como los procesadores de arquitectura RISC, se sustentan en la optimización del camino de datos mediante técnicas de segmentación (pipelining). La segmentación permite la ejecución simultánea de múltiples instrucciones dividiendo el proceso en etapas independientes. En una arquitectura clásica MIPS de 32 bits, estas etapas corresponden a Búsqueda (IF), Decodificación (ID), Ejecución (EX), Memoria (MEM) y Escritura (WB). No obstante, la superposición de instrucciones introduce riesgos de control cuando se ejecutan instrucciones que alteran el flujo secuencial del programa.

La Fase 3 de este proyecto cobra relevancia al incorporar el soporte para las instrucciones Tipo-J, centrándose en la instrucción de salto incondicional (j). A diferencia de las instrucciones Tipo-R o Tipo-I, el formato Tipo-J cuenta con un código de operación (Opcode) de 6 bits y un campo de dirección destino (Target Address) de 26 bits. Para reconstruir la dirección destino absoluta de 32 bits, el hardware realiza un desplazamiento lógico a la izquierda de 2 bits sobre el Target Address (multiplicando efectivamente por 4 para una correcta alineación de palabras en la memoria) y concatena el resultado con los 4 bits más significativos del PC incrementado actual (PC+4). Matemáticamente, la operación secuencial en el hardware se define como:

$$\text{Jump_Address} = (\text{PC} + 4)[31:28] \parallel (\text{Target_Address} \times 4).$$

Set de Instrucciones Seleccionado

Instrucción	Tipo	Sintaxis
Add	R	Add rd, \$rs, \$rt
Sub	R	Sub rd, \$rs, \$rt
Mul	R	Mul rd, \$rs, \$rt
Div	R	Div rd, \$rs, \$rt
Or	R	Or rd, \$rs, \$rt
And	R	And rd, \$rs, \$rt
Addi	I	Addi rt, \$rs, inm
Lw	I	Lw rt, inm(\$rs)
Sw	I	Sw rt, inm(\$rs)
beq	I	Beq \$rs, \$rt, etiqueta
j	J	J etiqueta

2. Objetivo

Objetivo General

Diseñar, describir e interconectar un datapath segmentado con arquitectura tipo MIPS de 32 bits capaz de procesar de manera concurrente las instrucciones lógicas, matemáticas y de control de flujo especificadas de la rúbrica.

Objetivos Particulares

- Modificar la lógica combinacional de la Unidad de Control principal para decodificar con precisión el Opcode de la instrucción de salto incondicional (6'b000010).
- Implementar físicamente el cálculo estructural de la dirección destino de 32 bits en la etapa de Decodificación (ID).
- Integrar un multiplexor lógico en la etapa de Búsqueda (IF) que intercepte dinámicamente la actualización secuencial del Contador de Programa (PC) cuando la bandera de control Jump se encuentre activa.

3. Desarrollo

La metodología implementada para la resolución de la Fase 3 consistió en la reestructuración completa del módulo maestro DPTR.v y el desarrollo acoplado del módulo de la Unidad de Control. El proceso técnico se dividió en los siguientes puntos clave:

1. Diseño de la Unidad de Control Combinacional

Se estructuró la lógica interna de UnidadDeControl.v mediante un bloque always @* que evalúa continuamente el código de operación. Para prevenir la inferencia indeseada de latches lógicos, se inicializaron por defecto todas las salidas de control en cero. Al detectar el Opcode de JUMP, la unidad aserta la señal Jump de manera inmediata, aislando el resto de las compuertas de escritura del procesador.

```
module UnidadDeControl(  
    input wire [5:0] op,  
    output reg MemToReg, RegWrite, MemToWrite, MemToRead, ALUSrc, RegDst,  
    output reg [2:0] ALUOp,  
    output reg Jump  
);  
    always @(*) begin  
        MemToReg = 1'b0; RegWrite = 1'b0; MemToWrite = 1'b0;  
        MemToRead = 1'b0; ALUSrc = 1'b0; RegDst = 1'b0;  
        ALUOp = 3'b000; Jump = 1'b0;  
        case(op)  
            6'b000000: begin RegDst = 1'b1; RegWrite = 1'b1; ALUOp = 3'b010; end  
            6'b000010: begin Jump = 1'b1; end // JUMP  
        endcase  
    end  
endmodule
```

2. Enrutamiento y Conexiones en el Top-Level (DPTR.v)

En el módulo principal DPTR.v, se incorporó una instancia formal del submódulo secuencial pc.v. Para realizar la concatenación espacial de los 32 bits de salto, se utilizó el operador de agregación nativo de Verilog: { id_pc_plus_4[31:28], id_instruccion[25:0], 2'b00 }. El flujo del datapath segmentado se gobierna mediante un multiplexor implícito parametrizado por la

```
pc_program_counter (
    .clk(clk),
    .addnew(next_pc),
    .address(dir)
);
assign next_pc = (Jump) ? jump_address : if_pc_plus_4;
assign if_pc_plus_4 = dir + 4;
```

[illegible]

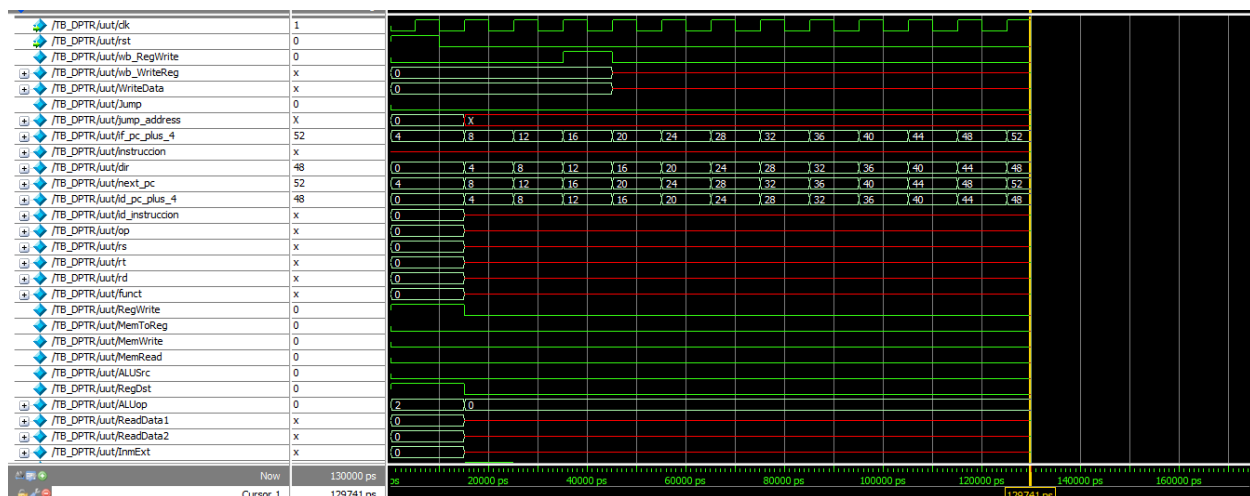
Implementacion y Verificacion de Algoritmo (MCD)

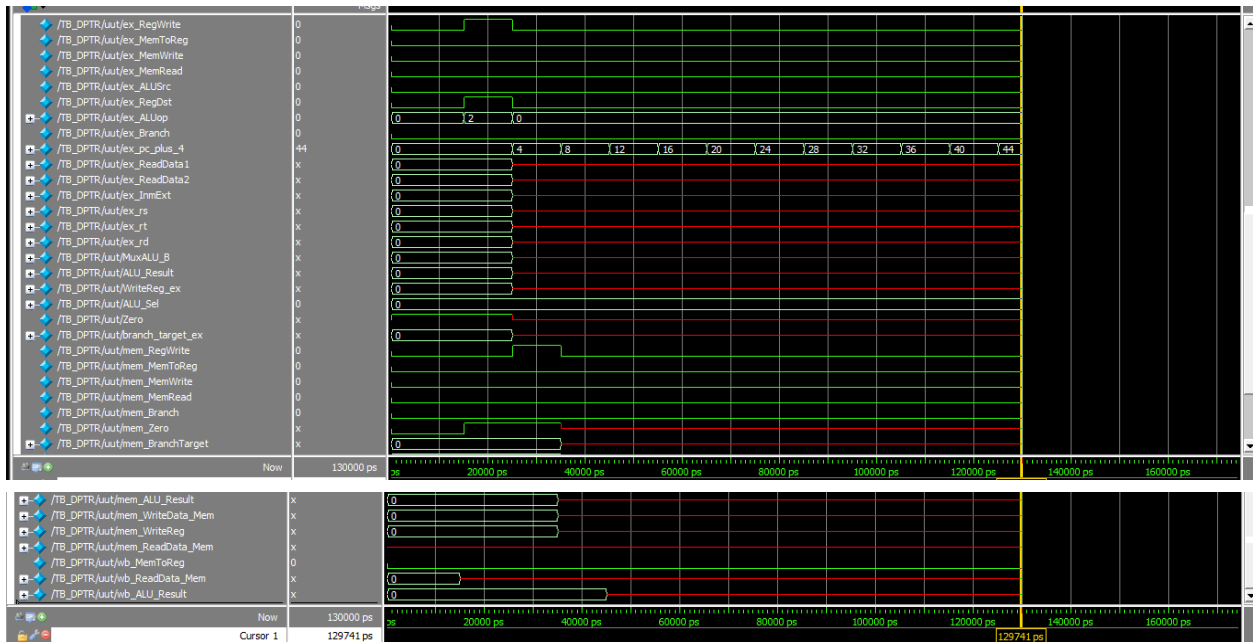
Para la fase final del proyecto, se verifico el correcto funcionamiento del procesador segmentado mediante la ejecucion del algoritmo del Maximo Comun Divisor (MCD) utilizando el metodo de restas sucesivas de Euclides.

Procedimiento realizado:

- **Codigo Ensamblador:** Se diseno el algoritmo en lenguaje ensamblador MIPS haciendo uso de instrucciones tipo R, I y J (addi, beq, slt, bne, sub, j, sw). El programa inicializa dos registros (\$s0 y \$s1) con los valores a evaluar.
- **Decodificacion a Lenguaje Maquina:** Se desarrollo un script en Python que actua como ensamblador. Este script lee el archivo .asm, identifica las etiquetas de salto relativas y absolutas, traduce cada instruccion a su equivalente binario de 32 bits segun la arquitectura disenada y genera un archivo de texto plano.
- **Carga en Memoria:** El archivo binario generado se integro al modulo de Memoria de Instrucciones utilizando la funcion de sistema de Verilog para carga de memoria, permitiendo al procesador leer el codigo maquina directamente en el inicio de la simulacion.
- **Simulacion y Resultados:** Se ejecuto la simulacion desde el Testbench general en ModelSim. El calculo finalizo de manera exitosa cuando los registros 16 y 17 del Banco de Registros igualaron su valor, demostrando el correcto funcionamiento del hardware de la Unidad de Control, el calculo de saltos y la Unidad Aritmetico Logica. Finalmente, el resultado del MCD se almaceno de forma correcta en la direccion 0 de la Memoria de Datos.

Simulación de MCD





5. Conclusión

La culminación de la Fase 3 del proyecto permitió consolidar de manera práctica los conocimientos abstractos adquiridos sobre el control de flujo en arquitecturas pipeline de alto rendimiento. Diseñar hardware capaz de romper la linealidad secuencial de la memoria a través de la instrucción de salto incondicional `j` puso en evidencia la elegancia de la filosofía RISC: delegar tareas complejas de decodificación a manipulaciones espaciales de bits y enrutamiento directo de cables sin sobrecargar la temporización del procesador. El desarrollo progresivo por fases facilitó de gran manera el aislamiento de errores de bus antes de proceder con el algoritmo final.

6. Referencias

1. Patterson, D. A., & Hennessy, J. L. (2013). Computer Organization and Design: The Hardware/Software Interface (5th ed.). Morgan Kaufmann.
2. Hennessy, J. L., & Patterson, D. A. (2017). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.